

ARCHITECTURES LOGICIELLES

Cours de préparation — S7 M1 Informatique (ILIA) — Option

Au-delà de MVC et des couches classiques : styles architecturaux, microservices approfondis, qualités architecturales, documentation d'architecture et patterns liés aux données — avec exercices corrigés.

Sommaire

1. Styles architecturaux
2. Microservices approfondis
3. Qualités architecturales et compromis
4. Documentation d'architecture
5. Architecture et données
6. Étude de cas
7. Exercices récapitulatifs

Chapitre 1 — Styles architecturaux

Tu connais déjà les couches et MVC. Ce chapitre élargit avec d'autres styles utiles pour structurer un système, notamment quand la logique métier doit rester indépendante des détails techniques.

1.1 Client-serveur

Le système est découpé entre un ou plusieurs **clients** (initient les requêtes) et un **serveur** (répond, centralise les traitements et données). C'est le style le plus répandu du web classique.

1.2 Architecture hexagonale (ports et adaptateurs)

L'idée centrale : isoler complètement le **domaine métier** (le cœur logique de l'application) de tout détail technique (base de données, interface web, API externe). Le domaine expose des **ports** (interfaces), et chaque technologie concrète est branchée via un **adaptateur**.

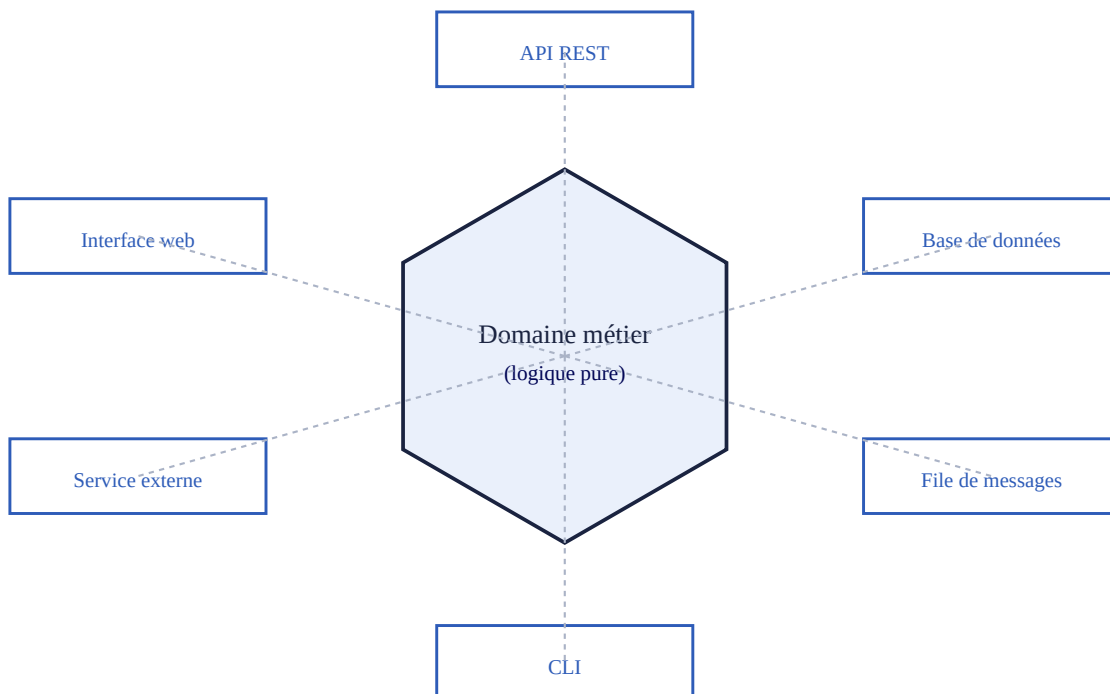


Figure 1.1 — Architecture hexagonale : le domaine au centre, entouré d'adaptateurs interchangeables

Bénéfice concret : on peut remplacer la base de données ou ajouter une nouvelle interface (CLI, API) **sans toucher au domaine métier**, et tester la logique métier sans aucune dépendance technique réelle (on utilise de faux adaptateurs pour les tests).

1.3 Architecture événementielle (event-driven)

Les composants communiquent en publiant et en écoutant des **événements** (ex. « CommandeCréée »), plutôt qu'en s'appelant directement. Cela découple fortement les composants : le producteur d'un événement n'a pas besoin de connaître ses consommateurs.

- **Avantage** : forte évolutivité — ajouter un nouveau traitement consiste juste à ajouter un nouvel abonné à l'événement existant.

- **Inconvénient** : plus difficile à déboguer et à tracer (le flux d'exécution n'est plus linéaire et explicite dans le code).

Exercice 1 — Identifier un style architectural

Une application de logistique doit déclencher automatiquement une notification, une mise à jour de stock, et un email de confirmation dès qu'une commande est validée — sans que le composant qui valide la commande ait besoin de connaître ces trois traitements. Quel style architectural choisir, et pourquoi l'architecture hexagonale seule ne suffit pas ici ?

Correction

Le style **événementiel** (event-driven) est le plus adapté : le composant de validation publie un événement « CommandeValidée », et les trois traitements (notification, stock, email) s'y abonnent indépendamment, sans couplage direct. L'architecture hexagonale organise surtout la séparation domaine/technique à *l'intérieur* d'un composant ; elle ne résout pas, à elle seule, le découplage *entre plusieurs traitements déclenchés par un même événement* — les deux approches sont d'ailleurs complémentaires et souvent combinées.

Chapitre 2 — Microservices approfondis

Le cours d'Ingénierie du logiciel a introduit les microservices comme alternative au monolithe. On approfondit ici les patterns qui rendent une architecture microservices viable en pratique.

2.1 API Gateway

Plutôt que le client appelle directement chaque microservice, une **API Gateway** centralise le point d'entrée : routage vers le bon service, authentification, limitation de débit (rate limiting), agrégation de plusieurs appels.

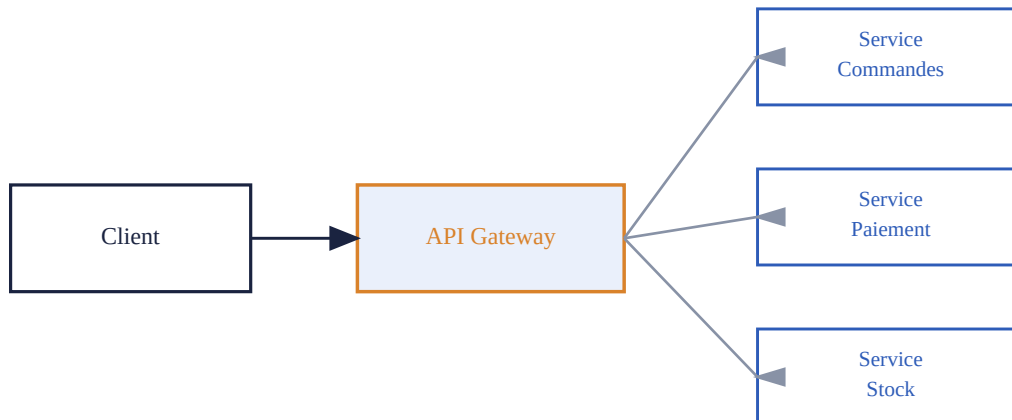


Figure 2.1 — API Gateway routant les requêtes vers les microservices adaptés

2.2 Circuit Breaker (disjoncteur)

Si un service appelé devient lent ou tombe en panne, un appelant naïf risque d'attendre indéfiniment et de propager la panne. Le pattern **Circuit Breaker** surveille les échecs répétés et « ouvre le circuit » temporairement : les appels échouent immédiatement (ou renvoient une réponse par défaut) au lieu de bloquer, le temps que le service se rétablisse.

État du circuit	Comportement
Fermé	Fonctionnement normal, les appels passent
Ouvert	Trop d'échecs récents : les appels échouent immédiatement sans solliciter le service
Semi-ouvert	Après un délai, quelques appels test sont autorisés pour vérifier le rétablissement

2.3 Découplage et communication asynchrone

Plutôt que des appels synchrones directs (service A attend la réponse de service B), on utilise souvent une **file de messages** (message queue) : service A dépose un message, service B le traite quand il le peut. Cela limite la propagation des pannes et absorbe les pics de charge.

Exercice 2 — Circuit Breaker en pratique

Un service de paiement devient très lent (timeout à répétition) suite à une panne réseau partielle. Sans Circuit Breaker, que risque-t-il de se passer pour les autres services qui l'appellent ? Comment le Circuit Breaker limite-t-il ce risque ?

Correction

Sans Circuit Breaker, chaque appel au service de paiement reste bloqué jusqu'au timeout, ce qui immobilise des ressources (threads, connexions) chez tous les appelants ; si le trafic est important, cela peut saturer les appelants eux-mêmes et propager la panne en cascade à tout le système (effet domino). Avec un Circuit Breaker, après un certain nombre d'échecs le circuit s'ouvre : les appels suivants échouent immédiatement (ou renvoient une réponse de repli), ce qui libère les ressources des appelants et évite la propagation en cascade, le temps que le service de paiement se rétablisse.

Chapitre 3 — Qualités architecturales et compromis

Une architecture ne se juge pas seulement sur ce qu'elle permet de faire fonctionnellement, mais sur des **attributs de qualité** (ou exigences non fonctionnelles, vues au chapitre Ingénierie du logiciel). Ces qualités sont souvent en tension les unes avec les autres.

Qualité	Définition
Scalabilité	Capacité à absorber une charge croissante en ajoutant des ressources
Disponibilité	Proportion du temps où le système répond correctement
Maintenabilité	Facilité à faire évoluer le système sans tout casser
Performance	Rapidité de réponse et débit du système
Sécurité	Résistance aux accès et usages non autorisés

3.1 Le théorème CAP

Dans un système distribué, le théorème CAP énonce qu'on ne peut garantir simultanément que deux des trois propriétés suivantes en cas de partition réseau :

- **Cohérence (Consistency)** : toutes les lectures renvoient la donnée la plus récente écrite.
- **Disponibilité (Availability)** : chaque requête reçoit une réponse (pas d'erreur), même si elle n'est pas la plus à jour.
- **Tolérance au partitionnement (Partition tolerance)** : le système continue de fonctionner malgré une coupure de communication entre certains nœuds.

En pratique : la tolérance au partitionnement est quasiment incontournable dans un système distribué réel (les coupures réseau arrivent). Le vrai choix architectural se fait donc généralement entre **cohérence forte** et **disponibilité maximale**.

3.2 Compromis (trade-offs)

Améliorer une qualité se fait souvent au prix d'une autre. Exemple : découper en microservices améliore la scalabilité et permet un déploiement indépendant, mais dégrade en général la simplicité de maintenance (plus de composants, plus de complexité opérationnelle) et peut dégrader la performance brute (latence réseau entre services).

Exercice 3 — Compromis architectural

Un réseau social doit continuer à afficher un fil d'actualité même en cas de coupure réseau partielle entre ses centres de données, quitte à afficher une version légèrement obsolète des posts. Quel choix fait-il, entre cohérence et disponibilité au sens du théorème CAP ? Justifiez.

Correction

Ce système privilégie la **disponibilité** au détriment de la cohérence stricte : il préfère répondre avec des données potentiellement légèrement obsolètes plutôt que de renvoyer une erreur ou de bloquer la requête en attendant une synchronisation parfaite entre centres de données. C'est un choix courant pour les fils d'actualité, où l'impact d'un léger retard d'affichage est bien moindre que celui d'une indisponibilité perçue par l'utilisateur.

Chapitre 4 — Documentation d'architecture

Une architecture non documentée se perd et devient difficile à faire évoluer collectivement. Deux approches courantes structurent cette documentation.

4.1 Le modèle des vues 4+1

Proposé par Philippe Kruchten, ce modèle décrit une architecture selon plusieurs vues complémentaires, chacune répondant à une préoccupation différente :

Vue	Répond à
Logique	Comment les fonctionnalités sont-elles organisées en classes/modules ?
Processus	Comment le système se comporte-t-il à l'exécution (concurrency, synchronisation) ?
Développement	Comment le code est-il organisé en modules/paquets pour l'équipe de dev ?
Physique (déploiement)	Sur quelles machines/serveurs le système est-il déployé ?
Scénarios (+1)	Cas d'usage qui relient et valident les quatre autres vues

4.2 Le modèle C4

Plus récent et très utilisé en pratique, le modèle **C4** (Context, Containers, Components, Code) propose 4 niveaux de zoom progressifs pour documenter une architecture, du plus général au plus détaillé.

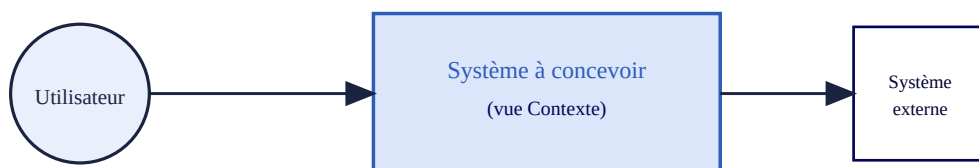


Figure 4.1 — Niveau Contexte du modèle C4 : le système vu de l'extérieur, avec ses utilisateurs et systèmes externes

Niveau C4	Contenu
1. Contexte	Le système dans son environnement : utilisateurs, systèmes externes
2. Conteneurs	Les grandes briques déployables (application web, base de données, API...)
3. Composants	Le détail interne d'un conteneur (modules, services internes)

4. Code	Diagrammes de classes ou équivalent, niveau le plus fin (souvent généré, pas dessiné à la main)
---------	---

Exercice 4 — Choisir le bon niveau C4

Vous présentez l'architecture d'un système à un client non technique, qui veut comprendre quels acteurs et systèmes externes interagissent avec le produit, sans entrer dans les détails techniques. Quel niveau du modèle C4 utiliser ? Et si vous deviez expliquer à un nouveau développeur comment les grands modules de l'application communiquent entre eux ?

Correction

Pour le client non technique : le niveau **1 (Contexte)**, qui montre le système comme une boîte unique entourée de ses utilisateurs et systèmes externes, sans détail d'implémentation. Pour le nouveau développeur qui doit comprendre l'organisation interne en grandes briques déployables : le niveau **2 (Conteneurs)**, qui montre par exemple l'application web, l'API backend et la base de données, et comment ils communiquent.

Chapitre 5 — Architecture et données

5.1 CQRS (Command Query Responsibility Segregation)

L'idée de CQRS est de séparer le modèle utilisé pour **écrire** des données (commandes) du modèle utilisé pour les **lire** (requêtes), qui peuvent être optimisés très différemment et parfois stockés séparément.

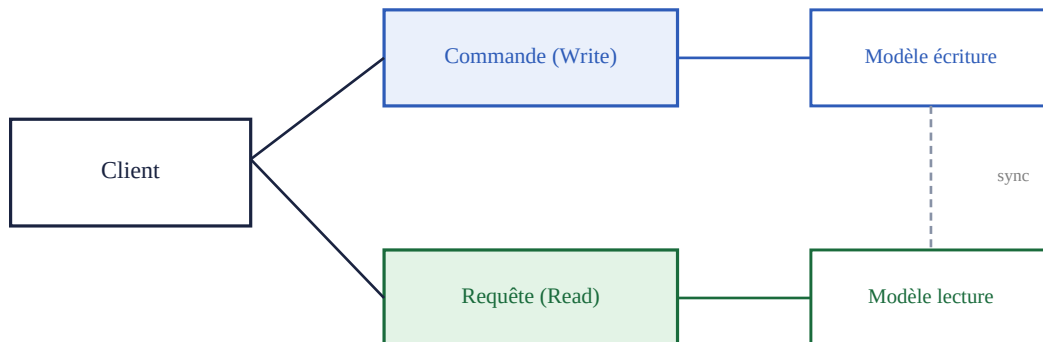


Figure 5.1 — CQRS : modèles d'écriture et de lecture séparés, synchronisés en arrière-plan

Pourquoi séparer ? Un modèle d'écriture privilégie la validation stricte et la cohérence des règles métier ; un modèle de lecture privilégie la rapidité d'accès pour l'affichage (souvent dénormalisé, pré-calculé). Les besoins sont différents, autant ne pas forcer un seul modèle à tout faire.

5.2 Cohérence forte vs cohérence à terme (eventual consistency)

Type de cohérence	Principe	Contexte typique
Forte	Toute lecture reflète immédiatement la dernière écriture	Transactions bancaires
À terme (eventual)	Les répliques convergent vers le même état après un certain délai	Fil d'actualité, compteur de vues

5.3 Réplication

Pour améliorer la disponibilité et la performance en lecture, les données sont souvent **répliquées** sur plusieurs nœuds. Un schéma courant : un nœud **primaire** reçoit les écritures, et les **répliques secondaires** (souvent en lecture seule) se synchronisent ensuite, avec un léger délai possible.

Exercice 5 — CQRS et cohérence

Dans un système CQRS avec cohérence à terme, un utilisateur vient de passer une commande (écriture), puis rafraîchit immédiatement la page de son historique de commandes (lecture). Quel risque cela peut-il poser, et comment l'atténuer côté interface utilisateur ?

Correction

Le modèle de lecture n'a peut-être pas encore été synchronisé avec la dernière écriture : l'utilisateur pourrait ne pas voir immédiatement sa commande dans l'historique, ce qui peut être perçu comme un bug. On peut atténuer ce risque en affichant un état visuel de confirmation immédiat côté client dès la validation de la commande (sans attendre le rechargement complet depuis le modèle de lecture), ou en affichant un indicateur « traitement en cours » le temps que la synchronisation se fasse.

Chapitre 6 — Étude de cas

Contexte : une plateforme de suivi de trafic maritime doit recevoir en continu les positions de milliers de navires (via AIS), les afficher en temps quasi réel sur une carte, et déclencher des alertes en cas de zone à risque — un contexte proche des applications possibles en ILIA (données environnementales et maritimes).

6.1 Analyse des contraintes

- Fort volume de données entrantes en continu (positions GPS de milliers de navires).
- Besoin d'affichage temps quasi réel côté utilisateur (carte interactive).
- Traitement asynchrone pour la détection d'alertes (zones à risque, collisions potentielles).
- Disponibilité importante, mais cohérence stricte moins critique qu'une transaction bancaire.

6.2 Choix architecturaux possibles

Besoin	Élément d'architecture pertinent
Ingestion de flux continu de positions	Architecture événementielle, file de messages
Détection d'alertes en arrière-plan	Service dédié, abonné aux événements de position
Affichage carte performant	Modèle de lecture optimisé (CQRS), éventuellement cache
Isoler la logique métier (règles de risque) du reste	Architecture hexagonale
Scalabilité face au nombre de navires	Découpage en microservices, réplication des services de lecture

Exercice 6 — Concevoir une architecture — étude de cas

En reprenant le contexte du suivi de trafic maritime ci-dessus : le service de détection d'alertes tombe temporairement en panne pendant une mise à jour. Que devrait-il se passer pour : a) les positions déjà reçues pendant la panne, b) l'affichage de la carte pour les utilisateurs connectés ? Justifiez avec les concepts du chapitre 2 et 3.

Correction

a) Si les positions sont publiées via une **file de messages** plutôt qu'envoyées directement au service de détection, elles restent stockées dans la file pendant la panne et seront traitées dès que le service redémarre — aucune donnée n'est perdue, seul le traitement des alertes est retardé.

b) L'affichage de la carte, s'il repose sur un **modèle de lecture séparé** (CQRS) alimenté indépendamment du service de détection, peut continuer à fonctionner normalement — seule la fonctionnalité d'alerte est temporairement indisponible, pas l'ensemble du système. C'est un exemple concret du compromis **disponibilité** du chapitre 3 : on isole la panne pour que le reste du système reste utilisable.

Chapitre 7 — Exercices récapitulatifs

Exercice 7 — Synthèse

Vous devez concevoir l'architecture d'une application de covoiturage.

- a) Quel style architectural utiliseriez-vous pour isoler les règles métier de calcul de prix des détails techniques (base de données, API de paiement) ?
- b) Quel pattern permettrait d'éviter qu'une panne du service de paiement externe ne bloque toute l'application ?
- c) Quel niveau du modèle C4 utiliseriez-vous pour présenter les grandes briques déployables (app mobile, API, base de données) à l'équipe technique ?

Correction

- a) Architecture hexagonale (ports et adaptateurs) : le domaine de calcul de prix reste indépendant de la base de données et de l'API de paiement.
- b) Circuit Breaker : après plusieurs échecs de l'API de paiement, le circuit s'ouvre et l'application peut réagir sans bloquer (ex. message d'erreur clair, nouvelle tentative différée).
- c) Niveau 2 (Conteneurs) du modèle C4.

Exercice 8 — Question ouverte

Expliquez en quelques phrases pourquoi il n'existe pas de « meilleure architecture » universelle, en vous appuyant sur la notion de compromis (trade-off) vue au chapitre 3.

Correction

Chaque choix architectural améliore certaines qualités (scalabilité, disponibilité...) au prix d'autres (simplicité, cohérence stricte, coût opérationnel). Une architecture jugée excellente pour un système à fort trafic et tolérant à une cohérence relâchée (un réseau social) serait probablement inadaptée pour un système bancaire exigeant une cohérence stricte à tout instant. Concevoir une architecture, c'est donc analyser les priorités réelles du projet (quelles qualités comptent le plus, dans quel contexte) plutôt que d'appliquer une solution universelle.